Process Report – StudyHelper

Institution:
VIA University College

Course & Semester:
SEP4, 4th Semester

Authors (Student Numbers):
Alexandru Savin(354790)
Cristina Aurelia Matei (354776)
Damian Michal Choina (354789)
Eduard Fekete (355323)
Jakub Maciej Baczek (354814)
Karina Rubahova (354565)
Mara-Ioana Statie (354536)
Marta Zrno (355351)
Piotr Junosz (355502)
Tymoteusz Krzysztof Zydkiewicz (355413)

Supervisors:
Erland Ketil Larsen
Joseph Chukwudi Okika
Kasper Knop Rasmussen
Markéta Tranberg

Date:
May 25, 2026

Character Count:
78,970 characters

TABLE OF CONTENTS

1. INTRODUCTION

Organizationally, the process was managed by following Scrum-like sprints and utilizing the task tracker Jira. The sequence of steps undertaken included development of the initial frontend structure and authentication, routing, integrating, improving usability, expanding the backend and security, and ultimately testing, preparing for deployment, and documenting the process. Sprint planning, sprint review, and retrospective activities helped assess progress and manage the process between sprints Throughout the project, certain technical and collaboration challenges arose. Some implementation details were subject to change, which was expected given the nature of the process, as the degree to which certain issues needed to be addressed was reconsidered, including frontend and backend integration, user authentication, device-server connection, and the features to be implemented. The project utilized Scrum sprints and the Jira task tracking system. The project included the following tasks: creating the interface structure and authentication system, developing routing and integrating functionality, improving usability, adding backend and security features, testing, deployment, and documentation. Sprint planning, sprint reviews, and retrospectives were used to evaluate progress and adjust work between sprints. During the project, the team encountered a number of challenges, including both technical and collaboration-related ones. Some aspects of the project turned out to be different than expected, particularly in relation to frontend and backend integration, user authentication, device connectivity, and the implementation of specific features.

2. GROUP WORK

2.1 Group Composition and Profiles

The group consisted of ten students with different technical strengths, but with a broadly shared regional background. Most members came from Central and Eastern Europe, including Poland,

Romania, Moldova, Latvia, Croatia, Slovakia, and neighbouring cultural contexts. This gave the group a relatively similar communication style and attitude toward practical problem solving, while still leaving enough variation in personality and work habits to make the collaboration interdisciplinary.

The shared cultural background made some potential collaboration issues easier to manage. Direct feedback was generally accepted well, and the group could discuss technical disagreements without needing a very formal hierarchy. At the same time, the project was large enough that some structure was necessary. Scrum Masters and sub-team leads helped coordinate deadlines and dependencies, but most day-to-day decisions were still made inside the relevant technical subteams. This made the hierarchy manageable rather than restrictive.

Time coordination was also helped by the fact that the group lived and studied in the same environment, even if individual schedules differed. Meetings, Discord discussions, and sprint planning made it possible to coordinate across frontend, backend, MAL, and IoT work. Different personalities also shaped the collaboration: some members were more structured and documentation-oriented, others were more experimental and implementation-focused, and others were strongest in debugging and integration. This mix was useful because the project required both planning and fast problem solving.

2.2 Roles and Contributions

The distribution of responsibilities was primarily based on technology areas: frontend, backend, MAL, and IoT. While initial plans for the distribution of responsibilities were developed early in the sprint planning process, the actual distribution gradually evolved throughout the semester as the project took shape.

Frontend-related tasks included dashboard development, localization, theming, profile features, responsiveness, calendar support, testing, and deployment preparation. Backend tasks focused on developing the authentication system, API endpoints, JWTs, and database interactions. MAL tasks included model-based predictions, dataset generation, and fitness prediction experiments. Tasks performed by the IoT team focused on sensor integration and connectivity.

Tasks were managed using tools chosen by each subteam. Some subteams used Jira to track sprint-related issues and their progress, while others used their own preferred tools for task management. GitHub branches and pull requests were used across all subteams for implementation-related work.

Member	Main Area	Main Contributions
Karina Rubahova	Frontend / Scrum	Localization, dashboard styling, theme system, testing, sprint documentation
Cristina Matei	Frontend / Backend Integration	Authentication integration, responsive improvements, profile functionality, backend/frontend synchronization
Marta Zrno	Backend	JWT authentication, API endpoints, database-related functionality
Eduard Fekete	MAL / DevOps / Documentation	MAL structure, data pipeline, session linearization, documentation automation, deployment support
Damian Michal Choina	IoT	Arduino firmware, Arduino-server communication, unit testing, instant-measure feature
Piotr Junosz	MAL / Scrum Master	Data search, data imputation with clustering, instant models
Jakub Maciej Baczek	Scrum Master / DevOps / Backend	IoT testing, IoT-facing API with unit tests, CI/CD for API and IoT
Mara-Ioana Statie	MAL / Documentation	Session model experiments, feature alignment, MAL tests, ML report sections
Alexandru Savin	MAL	Mock data analysis and preprocessing, model training and evaluation, API endpoints
Tymoteusz Krzysztof Zydkiewicz	IoT	Start/stop button feature, CO2 sensor integration, sound sensor investigation, database design

2.3 Team Dynamics and Collaboration

Communication took place via Discord, Jira, GitHub, and weekly project meetings. Discord was used for day-to-day communication, promptly exchanging requests, coordinating subgroup work, and sharing

progress updates. Jira helped manage sprints, assign responsibilities, track progress, and prioritize tasks throughout the semester. It's important to point that GitHub played a significant role in facilitating collaboration across the various project components, namely the frontend, MAL, and IoT implementations. Functions were isolated using branches and pull requests to reduce the likelihood of repository conflicts when multiple users were present at the same time. Coordination between subteams was essential when implementing shared branches that were dependent on each other. The decision-making process within our group project relied heavily on technical responsibility and subject matter expertise. Front-end decisions were made by front-end developers, while MAL and IoT decisions were overseen by the relevant subgroups. Broader decisions regarding architecture or processes were discussed and adopted collectively during meetings. Over time, communication within the project became more organized as integration issues became more complex due to the interactions between the front-end, back-end, MAL and IoT layers. Specifically, API conventions, authentication processes, and device/session synchronization posed challenges for coordinated work.

2.4 Conflict and Resolution

During the project, the team encountered some technical disagreements and collaboration issues. Sometimes, these issues arose due to inconsistencies in requirements between the frontend and backend. This could lead to duplication of effort, changes to the previously chosen structure, and the need for additional effort to integrate all components. Furthermore, during one of the sprints described in the documentation, an issue arose: one team member left the team and was absent for subsequent sprints, which impacted task distribution and presentation preparation. These issues were largely resolved through improved communication, discussions during sprints, the use of Jira for task management, and a clearer assignment of roles within the team. Team members began to more carefully consider architectural and integration decisions before collaborating on any changes. One lesson learned was the need for clear assignment of roles and improved communication in large teams consisting of multiple subteams.

2.5 Social Loafing and Accountability

One of the challenges the group faced was ensuring equal contributions from each member to a large project throughout the semester. Because the project encompassed various areas—frontend, backend, MAL, and IoT—the amount of each member's contribution varied and was sometimes difficult to estimate, especially given that some tasks were more complex than others. The group employed multiple strategies to ensure greater transparency, such as using Jira for task assignments, GitHub for commits and pull requests, sprints and sprint reviews, and direct communication between team

members. Task assignments were clear and continually updated during sprints. As the semester progressed, team members realized that problems could arise ddue to unclear responsibilities or poor communication, leading some team members to take on most of the integration and coordination work, unaware that they had been assigned other roles.This led to a greater focus in subsequent sprints on task allocation and decision-making before implementation began. This project clearly demonstrates that responsibility encompasses more than just task execution; it also encompasses communication, coordination, integration, and participation in meetings and sprints.

3. PROJECT INITIATION

3.1 Choosing the Problem Domain

The topic of StudyHelper emerged from a combination of personal relevance and technical ambition. Environmental conditions - temperature, noise, CO_2 levels, and lighting - were something every member of the group dealt with daily as students. The frustration of sitting in a noisy library, an overheated classroom, or a poorly lit dorm room and feeling unable to focus was a shared, concrete experience. That lived relevance made the problem domain feel worth solving rather than simply academically convenient.

The topic also satisfied the technical constraints of the semester well. It required all three sub-teams to contribute meaningfully: IoT for physical sensing, Machine Learning for predictive modelling, and Frontend for user-facing presentation. A problem that forced genuine integration between the three components was preferable to one where the teams could have worked in relative isolation and assembled results together at the end.

In hindsight, the choice was a good one from a motivation and scope perspective. The domain was grounded enough to avoid vague scope creep and broad enough to give each sub-team real engineering problems to solve. The main difficulty was that “study quality” as a concept is inherently subjective - something we did not fully appreciate until we were deep into the machine learning phase. Retrospectively, conducting a deeper analysis of the target variable’s measurability at the domain selection stage would have better prepared us for the shifts we later encountered in our machine learning approach.

3.2 Problem Statement Development

The problem statement went through several meaningful iterations before reaching its final form, and the evolution reflects how our understanding of the problem deepened over time.

The starting point was broad: we were interested in how environmental noise influences student focus. The early framing involved combining datasets of noise effects with labelled sound categories extracted from WAV files, attempting to correlate audio features with cognitive performance. This approach broke down quickly - "focus" cannot be directly observed or measured from sensor data alone, making it an unsuitable target variable for a supervised learning problem.

This led to a fundamental reformulation. Instead of predicting focus - an internal cognitive state - we shifted to predicting a user-provided **Study Suitability Rating**: a 1–5 score submitted by the student themselves after or during a session. This made the target variable concrete, collectible, and directly tied to user experience rather than inferred from proxies. The formal main problem question as documented in the project description became: "Can a distributed system of IoT sensors be used to measure, predict and display relevant environmental conditions to help students choose the most optimal study environment?", supported by sub-questions on which factors correlate with study efficiency, how students respond to changes, and how accurately effects can be predicted.

The shift from measuring focus to measuring suitability was the single most important intellectual decision of the initiation phase. It was also the decision that made the rest of the system - the session flow, the rating submission UI, and the data collection pipeline - actually possible to design in a coherent way.

3.3 Time Planning and Scheduling

Initial Planning We based our project timeline on Agile Scrum. Before writing any code, we used digital boards to map out our structure visually, which helped us turn abstract requirements into actual technical tasks. We started out with standard two-week sprints, planning to switch to one-week sprints later on.

Cross-Team Alignment in a Large Group To save time, we tried to keep full-team meetings to a minimum. Instead, the Product Owner and sub-team Scrum Masters just met as needed to handle cross-team scheduling and dependencies, rather than sticking to a rigid calendar. This made it easier to agree on interface contracts, sort out API issues, and manage integration, letting the rest of the

developers just focus on their sprint tasks. We did still hold occasional full-group meetings to share feedback and make sure everyone knew what was going on with the system as a whole.

Adapting the Schedule: Tighter Sprints and Standups During the project, we switched from two-week to one-week sprints. This shorter timeframe forced us to break tasks down into much smaller pieces right from the start. To keep up with this faster pace, we started doing daily morning standups. These quick syncs were great for keeping everyone accountable, maintaining momentum, and catching integration blockers before they could mess up the sprint.

Monitoring Progress: Burndown Charts We used custom Burndown charts to track our progress during sprints. They gave us a clear visual of how much work was left versus the time remaining. By watching the trend line, we could easily spot if we were at risk of missing our sprint goals. If the line flattened out, it was a cue for the Scrum Masters to step in and reprioritize things so we wouldn't drag too much technical debt into the next sprint.

Reflecting on Realism and Deviations Since our project involved three completely different tech stacks that needed to communicate flawlessly, we hit some bumps along the way. Because we cut down on cross-team meetings, some tasks that we initially marked as "done" actually failed during cross-group reviews. We also ran into delays because we had to keep tweaking and reorganizing our CI/CD pipelines.

Furthermore, we originally required at least four people to approve merge requests for common cross-team branches. We eventually had to drop that down to two just to speed things up and avoid bottlenecks.

3.4 Ethical Perspectives

Data Privacy and User Consent

The StudyHelper system collects sensitive data about when, and for how long students study, along with their perceived study quality ratings. This information could reveal personal habits, academic struggles, or vulnerable times. Ethical considerations included:

Data Minimization: The device transmitted only the environmental sensors necessary for the ML task (temperature, humidity, CO_2, light) — not geo-location, audio recordings, or biometric data.

User Consent and Transparency: No user concent is needed currently and the data is retained indefinitely, in production, a retention policy would be needed.

Database Security: TO DO

Future Consideration: If StudyHelper was deployed in a real school or university, GDPR compliance would be necessary and formal data processing agreements. The current system does not meet production-level privacy standards but demonstrates the team's awareness of these concerns.

Accessibility and Inclusivity

The team considered whether the system would serve all students equitably:

Physical Accessibility: The IoT device has a physical button for session start/stop and an onboard buzzer for alerts. Alternative input methods (e.g., voice commands, touch-less activation) were not implemented but could be added. The frontend is responsive and works on (phones, tablets TOBECHECKED), and laptops, addressing screen-size diversity.

Cognitive and Language Accessibility: The frontend supports English and Danish language selection. The rating scale (1–5) is simple and culturally neutral. However, no accessibility audit (e.g., screen reader testing, color contrast, keyboard navigation) was conducted. Future versions should undergo formal accessibility review.

Implicit Bias in Data: The ML models are trained on a mock dataset that may not represent all student populations. When real data becomes available, the team must audit the dataset for demographic imbalances (e.g., does the model work equally well for students with disabilities, students from different climate backgrounds?). Deploying a model without this analysis risks amplifying existing inequities.

Environmental and Societal Impact

Energy Consumption: The IoT device and cloud backend consume electricity. While StudyHelper is not intended for climate monitoring at scale, the team acknowledges that any deployed system should measure and minimize its carbon footprint — e.g., using renewable-powered hosting, efficient firmware that reduces network traffic, and device sleep states.

Societal Impact of ML-Driven Feedback: The system predicts study suitability and provides ratings to students. There is a risk that students might over-rely on automated predictions ("The app says I can't study now, so I'll procrastinate"). The team's ideas is that the system is a decision support tool, not an automated decision-maker — ultimately, users decide whether to study, move, or search for a new room.

Equity in Access: If StudyHelper were deployed at a university, access should be universal (not gated behind a subsription based service or limited to certain departments). The current prototype is to be kept open-source and freely deployable on any VPS, supporting this principle.

Recommendations for Future Deployment

If StudyHelper were to transition from a university project to a production system deployed at a real school or workplace:

- Conduct formal data protection assessments.
- Establish a data retention and deletion policy.
- Engage students and lecturers in ethical review before rollout.
- Retrain the ML model taking into consideration demographic differences.
- Provide transparency reports on data access and retention.
- Allow users to opt out of data collection while still using core features.
- Define clear governance for model updates to prevent silent changes to prediction behavior.

4. PROJECT EXECUTION

This section describes the development process, divided into our collective efforts and team-specific contributions.

4.1 Together Process

This section describes the development process, divided into our collective efforts and team-specific contributions.

As our project plan progressed, our system began to evolve into an integrated system requiring a high degree of interdependence across all components. Initially, at the beginning of the semester, subgroups were able to work more independently, as many components were implemented as mockupsor were in a work-in-progress state. However, as the development process progressed, the need for coordination increased significantly.

First, the frontend depended on several aspects of the backend, including authentication, API design, sessions, and sensor endpoints. Second, both the backend and MAL implementations relied on certain common conventions, such as JSON structures and predictive models. Thus, API contracts and data structures became integral to cross-team collaboration. To avoid collisions and conflicting implementations, the project relied heavily on branches and pull requests on GitHub. We used feature branches to allow teams to implement their solutions in parallel and then merge them into shared branches. However, the synchronization process sometimes became problematic due to changes in the backend and work on the frontend. Communication took place through Discord chats,sprint sessions, Jira tickets, and personal contacts with those responsible. The more complex the integration task, the clearer it became that even minor changes could simultaneously impact multiple aspects of the system.Among other interesting process observations, it's worth noting that the integration proved quite labor-intensive. Although everything worked individually, integrating all components, including the frontend, MAL, and IoT, proved to be a more complex task.

4.2 IOT Team Process

The IoT sub-team consisted of three members: Jakub Maciej Baczek, Damian Michal Choina, and Tymoteusz Krzysztof Żydkiewicz. None of the team members had prior experience with embedded systems programming in C, which meant the first sprint was spent largely on getting the environment working rather than writing application code. Setting up PlatformIO, understanding the AVR toolchain, and getting the first successful flash onto the Arduino took more time than anticipated, but it was a necessary foundation that paid off once development picked up pace.

4.2.1 Work Division

The team divided work based on voluntary preference rather than top-down assignment. At the start of each sprint, everyone stated what they were willing to pick up, and tasks were distributed accordingly. From previous SEP projects together the team had found that this approach works better than assigning tasks externally, since people are more invested in work they have chosen themselves. In practice, responsibilities emerged naturally: server communication and HTTP layer, sensor integration, main loop logic and testing - each had a clear owner without formal negotiation. The team maintained its own backlog rather than using a shared tool like Trello or Jira, which kept things lightweight for a three-person team.

4.2.2 Development Milestones

The project ran across seven sprints and progress was measured against three concrete milestones rather than abstract story points. The first was getting the Arduino communicating with the server at all — once that worked end-to-end, even with hardcoded values, the rest of the work had a clear foundation to build on. The second milestone was the running session feature, where the device could start a session, send periodic data with keepalive pulses and end the session cleanly via button press. The third was the instant measurement feature, where a button press triggered a one-off sensor reading and sent it for immediate ML prediction. Reaching each of these felt like a genuine shift in confidence for the team.

4.2.3 Testing and CI/CD

One deliberate process decision was investing in automated testing and a CI/CD pipeline early in the project. Because the firmware runs on hardware, the team chose to run unit tests on the host machine rather than the device, using the Unity testing framework with FFF-generated fakes to replace AVR hardware dependencies. This allowed testing of application logic — server communication, session state, response parsing — without needing the physical Arduino present. A GitHub Actions workflow was set up to compile the firmware, run the full test suite, and upload coverage reports on every pull request. In practice the pipeline needed some adjustment as the project evolved, and having it fully configured from the very beginning would have saved some rework. That said, having it in place at all caught several issues before they reached the hardware and gave the team confidence during integration.

4.2.4 Integration with Other Teams

The API contract between the IoT team and the rest of the group was discussed at the start of the project, which gave a clear target for the communication layer. However, the contract evolved over time as certain features turned out to be impractical and requirements became clearer on both sides. Endpoint paths and response field names changed at points during development, which required adjustments to the firmware. These were manageable but reinforced the value of treating the API contract as a living document that all teams need to stay aligned on throughout the project, not just at the kickoff.

4.3 MAL Team Process

The Machine Learning and API (MAL) development followed an iterative path from data exploration to pipeline implementation. The following notes capture the key decisions and observations made during this process.

4.3.1 Mock Data Search and Goal Refinement

Initially, the focus was on how environmental noise influences focus. We attempted to combine datasets of focus-related noise effects with labeled sound categories from WAV files, extracting frequencies and loudness. However, we realized that "focus" cannot be measured directly. Consequently, we narrowed the goal to predicting a user-provided **Study Suitability Rating**, which made the target variable more grounded in user feedback.

4.3.2 Data Quality and Correlation Analysis

Further investigation led to the elimination of several datasets that appeared synthetic. We observed that in some candidate datasets, the distributions of features like humidity, noise, and light were suspiciously uniform, suggesting algorithmic generation rather than real sensor collection.

We also analyzed feature correlations as part of our validation process. Healthy datasets showed natural physical correlations (e.g., CO2, temperature, and humidity), while suspicious datasets exhibited either zero correlation or extreme overfitting potential. We decided to merge diverse datasets to create a more robust mock dataset.

4.3.3 Advanced Imputation Strategy

To handle missing values after merging, we implemented a sophisticated approach using the **MICE (Multivariate Imputation by Chained Equations)** framework. We enhanced this by:

- **Clustering**: Using k-means to find environment types (e.g., sessions made with high sun exposure vs. sessions made in labs).
- **ExtraTrees Estimator**: Modeling complex non-linear relationships.
- **Variance Modification**: Including natural distribution variance to avoid "flat average" imputation.
- **Global Median Fallback**: Used for extreme sparsity to prevent bias.

4.4 Frontend Team Process

The frontend development process changed significantly throughout the semester because the frontend depended heavily on MAL, and IoT integration. Many frontend decisions that seemed simple at the beginning became more complex once real API responses and session logic were introduced.

4.4.1 Initial Frontend Structure and Routing

In the early stages of the project, the frontend team focused on creating the core structure of the application in React. Initially, the developed modules focused on routing, navigation, login/registration screens, layout templates, and dashboard placeholders.

One of the key early decisions regarding the frontend was to divide the application into pages, components, services, and contexts to avoid bundling all the logic into large React components.

At this point, all frontend-related functions relied on mock data, as the backend API had not yet been fully developed.

4.4.2 Dashboard Development and UI Iterations

The dashboard emerged as the most frequently changed feature on the frontend throughout the semester. Initially, the dashboard was just filled with dummy cards and static information, however, further iterations of the dashboard included:

- Dynamic sensor values
- Recommendation cards
- Session-related information
- Prediction-related UI
- Responsive layouts
- Better loading and empty states

There were several parts which were reworked multiple times as there were changes in backend response and data structures when integrating. Some UI states which appeared to be straightforward at first became more complex after adding real data handling functionality.

The front end components had to support:

- Missing values

- Delayed responses
- Loading states
- Session synchronization
- Empty datasets
- Changing API structures

This showed that frontend complexity increased significantly once mocked data was replaced with real integration.

4.4.3 Localization and Theme System

Localization support was introduced during later frontend iterations. The application implemented English and Danish translations using React Context and centralized translation objects.

At the same time, a dark/light theme system was added using ThemeContext and localStorage persistence. Initially, theme handling looked relatively simple, but later many components needed additional refactoring because cards, charts, navigation, popups, and recommendation elements all needed to react dynamically to theme changes.

One problem discovered during this phase was that UI consistency became harder to maintain once localization and themes affected nearly every component in the system.

4.4.4 Backend Integration Challenges

Backend integration was one of the most challenging parts of the frontend process. At the beginning, many frontend pages used mock data or temporary localStorage logic, but later these parts had to be replaced with real API communication. This affected login, register, profile, dashboard data, calendar events, device connection, session handling, and rating submission.

One repeated challenge was that frontend work often depended on backend endpoints that were still changing. For example, authentication changed from a simpler token/localStorage approach to cookie-based authentication, which meant that protected frontend requests had to include credentials and the login/logout flow had to be adjusted. Similar changes happened around profile data, calendar data, connected devices, and session IDs.

The dashboard integration was especially difficult because it depended on both backend and IoT behavior. The frontend could not create the real study session itself, because the session lifecycle belonged to the IoT/backend flow. Instead, the frontend had to check whether a connected device already had an active session and then attach the dashboard and rating popup to that session. This required more coordination between frontend, backend, and IoT than originally expected.

Deployment and local testing also created some confusion. The frontend used `/api` routes so it could work behind Nginx and Coolify, but local Docker testing and deployed testing needed different API targets. This led to problems where requests were sometimes sent to the wrong backend or returned HTML instead of JSON. The solution was to make the API proxy configuration environment-based, so local Docker and deployment could use different targets without changing React code.

Overall, backend integration showed that frontend development was not only about building pages. A large part of the work was understanding API contracts, authentication, data ownership, deployment configuration, and how changes in other teams affected the user interface.

4.4.5 Scrum Workflow and Task Distribution

A sprint-based methodology was used during frontend development, leveraging the Jira task management system. Tasks were broken down according to the requirements of a specific sprint and the needs of the frontend at that stage of the process.

Planning served as a guideline for determining the implementation order, and a review and retrospective process was used to assess any integration issues, UI issues, and unfinished work on the frontend.

Given that frontend processes were heavily dependent on backend processes, IOT and MAL, priorities could shift during implementation. Therefore, constant coordination between frontend and backend specialists was crucial.

4.4.6 Frontend Testing and Stabilization

Interface testing was only implemented during the final sprint due to integration difficulties. Tools such as Vitest, jsdom, React Testing Library, and the user-event Testing Library were used for interface testing.

These tests primarily covered:

- Dashboard rendering
- Theme switching
- Localization behavior
- localStorage persistence
- Session rating flow
- User interaction behavior

An important observation made during testing is that automated interaction tests helped identify frontend issues that would not have been detected by manual testing.

Overall, the frontend development phase demonstrated how frontend implementation relies heavily on communication and API quality, as well as the collaboration of various subteams.

4.5 Application of Methods and Theories

4.5.1 IoT

For the IoT team, the main engineering method was turning general project requirements into concrete device behavior. The Arduino was not just a standalone prototype, but part of the full StudyHelper system. It had to read temperature, humidity, CO2, and light, start and stop sessions with a button, send data to the backend, and trigger the buzzer based on prediction results.

The modular firmware structure worked well for this. Separating areas such as server communication, Wi-Fi handling, sensors, buttons, timers, display status, and buzzer logic made the code easier to understand and test. This was especially useful because embedded C can become messy quickly if too much logic is placed directly in the main loop.

Testing was also a valuable method. Since running every test on the physical Arduino would be slow and unreliable, we used Unity and FFF fakes to verify firmware logic on the host machine. This allowed us to test session handling, response parsing, and server communication without constantly depending on the hardware. The CI/CD pipeline further supported this by compiling the firmware and running tests automatically on every pull request.

There was a learning curve early on. PlatformIO, AVR tooling, embedded C, timers, and hardware communication were all new to the team, which made the first sprint slower than expected. Once the setup and project structure were in place, development became much smoother.

Overall, the methods fit the problem well. Modular design and automated testing made the firmware more maintainable and testable, while requirements analysis helped clarify how the IoT device should interact with the backend and frontend. One area that fell short was API stability: endpoint names, response fields, and session behavior changed during development, causing rework. This showed that API contracts need to be treated as living documentation and kept updated throughout the project, not just agreed upon at the start.

4.6 Challenges During Execution

4.6.1 IoT

One of the main technical challenges for the IoT team was the sound sensor. Noise level was originally considered an important environmental factor for study conditions, but the available sensor did not work well for this purpose. It reacted mostly to sudden or close loud sounds, but it did not reliably measure background noise in a room. Because of this limitation, using it in the final system would have produced misleading data, so the team decided not to include sound measurement in the active firmware.

Another challenge was the development and testing setup around PlatformIO. While PlatformIO was useful for flashing and building the Arduino firmware, it was difficult to control exactly which files should be included in specific test builds. This became a problem when trying to test isolated modules, because PlatformIO tended to force its own build structure instead of giving us full control over compilation. To solve this, the team created Makefile-based test builds and compiled the unit tests with GCC. This made the testing process more predictable and allowed us to use Unity and FFF fakes without depending on the full embedded build every time.

The most difficult technical area overall was networking between the Arduino, the Wi-Fi module, and the backend API. Some of the Wi-Fi-related drivers and communication layers were harder to work with than expected, especially because errors could come from several places: UART communication, Wi-Fi module responses, HTTP formatting, backend availability, or timing issues in the firmware. The team responded by separating the networking logic into smaller modules and testing server communication as much as possible outside the physical device. In the end, the device was able to register, start sessions, send keep alive pulses, transmit sensor data, and receive prediction responses, but this part of the system required more debugging than most other IoT features.

4.6.3 Frontend

A challenge that the frontend team encountered was with a teammate who was not contributing equally, was missing meetings, did not want to participate in a group presentation and refused to read and reply to messages. Unfortunately, the situation escalated and they left the group.

Additionally, poor coordination with the other subteams presented obstacles. For example, during the creation of the session and preferences functionalities, it was unclear how data travels, and if frontend should trigger the device. Also, we did not plan the development of backend properly, so there were

some issues with deciding on which language it would use. These isses were resolved by holding a meeting with the whole group and coming to a decision and mutual understanding.

4.7 Deviations from the Plan

Several parts of the project changed compared to the original plan. Some changes were intentional decisions after the team understood the problem better, while others were caused by technical limitations or integration issues. Overall, the project moved from a more simple frontend prototype with mock data toward an integrated system connected to backend, IoT, ML, and deployment. The changes made the project more complex, but they also made the final solution more realistic.

4.7.1 IoT

The IoT part changed in several ways compared to the original plan. The biggest removed feature was sound measurement. Noise level was originally considered relevant for study conditions, but the available sound sensor only reacted reliably to sudden close sounds, not normal background noise. Because of this, including it would have produced misleading data, so it was removed from the active firmware.

The device interaction also changed. At first, the plan did not include both a start/stop session button and an instant measurement button, but these were added because they made the prototype more practical. The start/stop button gave the device clear control over the session lifecycle, while the instant measurement button allowed a quick reading and prediction without waiting for a full session.

Some technical choices also changed during implementation. The backend moved from an earlier C# idea to Python FastAPI because it fit the final architecture and MAL integration better. The buzzer logic was adjusted from activating only at study quality 1 to warning for generally poor conditions. The IoT tests were also moved from the PlatformIO test runner to Makefile-based GCC builds because PlatformIO did not give enough control over isolated test compilation.

Finally, HTTPS was considered for device-to-backend communication, but the final prototype used HTTP. Based on research, HTTPS was judged difficult to implement reliably on the available Arduino and Wi-Fi setup within the project scope. Since the transmitted data was mainly environmental sensor readings, HTTP was accepted for the prototype, although HTTPS would be necessary in a production version.

4.7.2 MAL

For MAL team, several deviations from the plan took place during the process of work. Initial plan was to mock the data while developing the models pipelines so that we can start experiments before getting real data from the sensors. After the meeting with supervisor and strict feedback we knew that we cannot generate the data in order to mock it, but we have to search for available datasets on the internet. Another deviation from the plan was that we wanted and thought that the only data we are going to use is going to be the real one collected from iot/frontend (the mock one only for experiments before real data coming in). Unfortunatelly, we had to change the plan again because the amount and variety of data was not sufficient. So plan changed to search for even more available, related datasets on the internet, which was a real challange since the datasets never had all required features. Than we had to glue those sets which also was not existing in the initial plan. The last significant change of plans was regarding the ml model. Initialy we were developing the model which were learning based on sessions but another type of model was needed as well - for instant predictions where the prediction is just based on the current sensor values and user raiting.

4.7.3 Frontend

The session flow also changed during the project. Earlier frontend ideas assumed that the user could start and stop a study session directly from the web application. During integration, the team clarified that the real session lifecycle should be handled by the IoT device and backend. Because of this, the frontend Start Session button was changed to attach the dashboard to an already active IoT session instead of creating a new session.

The frontend also changed more than expected. At first, many pages used mock data and simple local state. Later, these had to be adapted to real backend responses, authentication cookies, protected requests, device connection, active session checks, and rating submission. This caused extra rework, especially on the dashboard and profile page, but it made the application closer to the final system.

Deployment also required adjustments. The team originally worked mostly with local development, but later the application had to work through Docker, Nginx, and Coolify. This created some issues with API routing between local and deployed environments. The team adapted by using `/api` routes and environment-based configuration so the frontend could work in different environments.

4.8 Use of Tools and Technologies

Visual Studio Code was used as the development environment, since it supports various languages, extensions and tools. GitHub was crucial for team collaboration, since it enabled us to share code. It provided us with version control, ability to create pull requests, branch development style and GitHub Actions workflows for automated builds.

Jira was used to delegate tasks, split them up into sprints, keep track of deadlines, meetings. It helped structure the workflow and manage responsibilities. Discord was used to share messages, hold meetings and exchange feedback.

Figma was used mostly for design. For example, for diagrams and wireframes. But it also served as a note taking site for ideas, goals, group availability, etc.

Overall, the selected tools and technologies that were used throughout the project were effective and useful. They simplified collaboration, which at times was hard.

4.8.1 IoT

For communication, the IoT team mainly used Discord for quick discussions, progress updates, and solving problems between meetings. Trello was used for task management, which worked well for a small three-person subteam.

The firmware was developed in C for the ATmega2560 Arduino setup, using Visual Studio Code and PlatformIO for development, building, and flashing. YAT was used to inspect serial communication, especially when debugging UART messages and Wi-Fi module responses.

For testing, we used Unity with FFF fakes to test firmware logic without always depending on the physical Arduino. Since PlatformIO was not flexible enough for some isolated test builds, Makefile-based GCC compilation was also used. GitHub and GitHub Actions supported version control, pull requests, automatic firmware compilation, and test execution.

4.8.2 MAL

The MAL team primarily used Python as the core programming language for both data analysis and backend development. Jupyter Notebooks were extensively used during the experimental phases for data cleaning, MICE imputation, and testing various machine learning algorithms. Libraries such as

Pandas and NumPy were essential for data manipulation and clustering, while Scikit-Learn was the primary framework used for training and evaluating both the instant and session-based models (including Random Forest Classifiers and Neural Networks). Matplotlib and Seaborn were utilized to visualize data distributions and generate confusion matrices for evaluating model performance.

For integration and deployment, the team utilized FastAPI to expose the trained models through a robust, asynchronous REST API. The API and data pipeline were containerized using Docker to ensure a consistent environment from development to production. For quality assurance, `pytest` was used to write and execute unit and integration tests covering the machine learning pipeline and API endpoints, which were automatically run through GitHub Actions CI/CD workflows.

5. PERSONAL REFLECTIONS

Piotr Junosz

Fourth semester work was so far the most challanging in terms of communication, expectations, performing and coordination within the big 11 people group. It was my first time trying to develop a solid project in such a big team consisting of 3 subteams, trying to make sense out of the important assigment.

The first significant change I noticed was how the motivation was working. It is well described by Victor Vroom's book "Work and motivation" where he says that effort is tied up to expectations (Vroom, 1964). So if you believe your hard work will result in a finished project, you are highly motivated, but if you look around and see that other people are not working, your expectation of success drops, and your motivation completely collapses. This was seen during this semester project period and led to periods of members not motivating themself as much as in previous semesters. Personally, this made the process of developing the system a lot less exciting.

Another correlated phenomenon that occured is called diffusion of responsibility where a person is less likely to take responsibility for an action when others are present (Darley & Latane, 1968). This has grown even more because the group had 11 people and it is easier to "hide" in this group than group consisting of 4-5 people. The larger a group becomes, the less individual work each member will take. This combined with problem of motivation made me realised few times that I have to work more as

individual in order our project to succeed but then I understood that alone I cannot finish it fully which made me loose motivation and not enjoy the process of work on this project.

Having experienced those problems, I think what makes a big difference between a real job team and what i call “company simulation” of our big group is the role of a manager or a leader. Even though we were using Scrum roles such as Product owner and Scrum master, we were missing someone whos only responsibility would be focusing on connecting subteams work and generally leading the whole project process. I believe this could be a solution to at least the problem with motivation.

Karina Rubahova

At the beginning, I was actually interested about this project because it looked much bigger and more realistic than to previous semester projects. Earlier projects were usually smaller and easier, but this time we had frontend, backend, MAL, IoT, authentication, predictions, sessions, and many connected parts working together. It was interesting, but also stressful. In my opinion, the most challenging in project was that we had to work in a large group. Because the entire group was divided into several subgroups, it sometimes felt as if each of them was acting as an independent unit rather than part of the main team.

One of the biggest technical challenges for me was that backend functionality and API structures were changing quite often. Sometimes frontend work was already finished, but later had to be changad again because authentication flow, JSON responses, or backend logic changed. Merge conflicts also became stressful sometimes, especially during later sprints when more people worked on connected functionality at the same time.

At the same time, I think this project helped me learn realistic software development. I improved my understanding of React architecture, Git workflow, large project structure, frontend/backend integration, and automated testing. Before this project, I did not really understand how important frontend testing can become in larger systems. Working with Vitest and React Testing Library helped me understand how tests can detect problems that are difficult to notice manually.

I think one of my stronger contributions during the project was helping keep frontend work moving forward even during unstable integration periods. I worked a lot on frontend testing, sprint documentation, localization, theme-related functionality, and frontend fixes during integration. I felt that my work was important and visible in the final system.

Even though the project was stressful at times, I still think it was a valuable experience. I learned much more about communication, integration, coordination, and realistic software development than in previous semester projects. The project also showed me that large systems are not only about writing

code — a lot of the work is connected to communication, synchronization between teams and adapting to changes during development.

Alexandru Savin

Data vs. Models, what drives better outcomes? Thoughout the project me, being part of MAL team enjoyed analysing this question and I came out to the conclusion that even a relatively simple model can achieve pretty high accuracy, while a more powerful model cannot compensate from a lack of data.

[WIP]

Damian Michal Choina

This project was my first time writing embedded systems code in C, and adapting to manual memory management, hardware registers, and interrupt-driven thinking took real adjustment, though once past that initial curve the work became genuinely enjoyable. Setting up the CI/CD pipeline at the scale was also entirely new to me, and seeing it catch issues automatically made the investment feel worthwhile. What made the technical challenges manageable was the sub-team dynamic — having worked together in previous SEP projects meant skipping the usual early friction and moving straight into productive collaboration. Looking back, the main things to do differently would be establishing a minimal end-to-end path in the first week, writing tests alongside the code rather than deferring them, and setting up the CI pipeline from day one.

Eduard Fekete

Learning Outcomes

This take may sound paradoxical but I believe now in some power of pointless meetings. I believed that if there is no agenda, then there is no reason to meet. I can see that both supervisor meetings with merely explaining what we have done would be great early on and also some team meetings when we would just talk about what we have done, alternatively with some team-building aspect could have had a large impact.

I also learned that information should be structured in a more hierarchical way. Strongest project-shaping decisions should always be visible in a simple format, more details can be linked to that and so

on... I often referenced details and technicalities by saying “there is a file in the repo” or “you can see it on Figma”, which in my head made sense, as I could fully orient in these environments but I could see towards the end that a large failure on my side as a Product Owner was that many people did not have a clear idea of what was where.

I also learned that there is a whole parallel universe going on besides school and work. It would be a nice utilitarianistic utopia to just let everyone deal with their own lives but it literally would improve collaboration to know what’s going on with people besides the project. Sometimes I just feel like an hour of talking and 2 hours of work could have been more efficient than even 4 hours of working.

Contribution and Role

I was the Product Owner of the project, which meant that I was responsible for defining the product vision, managing the backlog, and ensuring that the team was aligned with our goals. I took this role seriously and tried to be as proactive as possible in communicating with the team and the supervisors. I tried to develop and communicate a strong product vision since the beginning, maintaining many of the diagrams and overall documentation. On top of that, I often ended up being “the person” for people who needed to quickly check on various decisions about to be made in the project, meaning I often found a lot of work even besides my typical work in my scrum team.

Although I believe that I have done my part in the project, I also feel that the way I approached everything could have been different. Different ways and channels of communicating all the information, better structuring of the information, and most importantly listening more to the team as often finding the best solution is not just about the problem itself but also about the team solving it.

I became a product owner mostly because this role stuck with me from previous projects and there was nobody else interested in it this time. A lot of trust and expectations were put on me, which I did my best to meet but I also can see that if I put more focus on the project, I could have achieved more. I remained my largest critic throughout the project, and I would love to have a peaceful mind concluding that I didn’t get negative feedback, but how hypocritical would that be to say right after mentioning I didn’t ask for enough feedback from the team?

Challenges and Growth

The most challenging aspect of the project was navigating the uncertainty of the situation overall. I can see many of my classmates, including myself, being affected by uncertainty and general confusion around the integration of artificial intelligence in learning and the industry in general. This affects motivation, and often prompts questioning of what is the point of our work and how should we

approach getting help on such projects? How is it different talking with supervisors, searching around StackOverflow, asking for help in the team and asking AI?

Believing I could answer this well in a small reflection section in a random semester project would be ambitious and my confidence in solving world problems casually as part of my school curriculum is not that high. However, as I am expected to show growth in this section, and I am not planning on uploading the history of my personal scale, I would summarize my way of overcoming this and dealing with the situation in a single quote - “talk with people”. It does not matter if this is about seeking supervision, lacking motivation to open the project rather than scrolling through social media or just generally feeling lost in the project. Almost always the answer is to talk with people, more is usually better.

Another challenge was the lack of a good structure and a highly related problem of a lack of transparency. The scrum teams were allowed to choose their own ways of working and they did. Frontend managed via Jira, IoT relied on Trello, MAL got immersed in Figma. On one hand - scrum utopia; every team choosing their own personal efficient way of working. On the other hand - a complete mess and a lack of transparency. The confusion about what is doing what, mostly across teams and the amount of accusations and paranoia that arised from that is unprecedented. I once completed a course about remote team leadership by Chris Croft, and among many of the takeaways was the importance of transparency and the feeling of inclusion in the project. What a mistake of not treating this project as a remote team. I wish I could have made everyone feel more included and more onboard about what is going on. Alignment meetings and a sea of DMs was not enough.

Yet another beautiful challenge I had to keep for the end was the problem with getting everyone onboard for specific technologies. I personally had my VPS with Coolify installed as a sort of “hammer for all nails” solution. I knew it might have not been the best but nobody else had better ideas available and I saw that even going with Azure, AWS, … would have a similar learning curve, this time with what seemed to be even less people knowing what is going on. I saved this challenge for the end to leave on a more positive note as I believe we have managed this well. People seemed to have researched about Coolify, everyone was given admin access since the beginning and every time I wasn’t available, various people decided to fix things on their own rather than to wait for me. And this was exactly what I wanted it to be. I am far from calling this perfection, but I am satisfied with what I could have influenced and what I have achieved on my side.

Reflection Forward

It is difficult to point out in several small chunks the key takeaways from such a project. On one hand, not having anything to point out suggests that a proper reflection has not been done, and on the other,

allowing myself to summarize my experience into bullet points would be a massive oversimplification. However, these are my strongest points to be considered in future projects:

Being data-driven inwards, not just outwards - on the previous semester project I was able to work in a highly data-driven fashion and I considered it the peak of what can be achieved in terms of acting rationally on such projects. In the future, I will focus on not just scanning the business aspects of features, and similar but also to understand better the team itself, skills, capabilities, obstacles, and so on.

Talk to people - I have very strongly outlined the idea behind this above but again, the best solution for uncertainty, and overall "human problems" is to talk to "humans". Seeking supervision, understanding the situation beyond the project, aligning, and so on.

Being remote is not about the distance - Regardless of how inaccurate this point sounds, yes, sometimes even though we are in the same class, we communicate in highly asynchronous ways and as it is usual with Software projects, a lot is managed digitally. I will treat more projects in this way as disregarding the specific aspects of such projects can be problematic and the gap will reveal itself in nasty ways.

Mara-Ioana Statie

Before this project, machine learning theory feelt abstract. Some concepts I understood in class, but not always as engineering decisions. During this project, I got to apply those ideas more directly. Working with imperfect datasets, creating said datasets, trying different models, checking whether results were meaningful, and seeing how the ML service had to fit into the rest of the system helped me understand the theory much better than only reading about it.

I also learned that communication becomes much more difficult when the team is large and split into sub-teams. We did have full team meetings, and they were useful, but looking back I wish we had more of them, especially during the middle of the project. At the same time, I can see why this is complicated in a group of this size. Full team meetings can become long, and hard to schedule, and not every topic is relevant to every member. Still, I think more regular alignment meetings would have helped us notice dependencies earlier and made it easier for everyone to understand how the IoT, MAL, frontend, and backend parts were developing together.

One thing I think worked well was that we stayed consistent with Scrum meetings throughout the project in my subteam. Even when progress was uneven or some tasks were unclear, the routine of meeting, discussing what had been done, and identifying blockers helped the MAL team keep moving. This matched the purpose of Scrum events described by Schwaber and Sutherland (2020): they are not

just meetings for the sake of meetings, but checkpoints that create transparency and give the team a chance to inspect and adapt.

For future projects, I would like to be more active in asking for cross-team updates earlier, not only when it becomes urgent. Overall, this project made me more confident in applying machine learning theory in practice, and it also showed me that technical progress depends heavily on communication, especially in large teams.

Marta Zrno:

I thoroughly enjoyed working on this project, specifically the division into subgroups. These subgroups allowed us to have clear roles and responsibilities during the development process. It helped emulate a semi-professional working environment, which I consider to be valuable preparation for future projects in the software industry. On the other hand, organization was chaotic at times, and it was hard to get on the same page, especially among the subgroups. I personally didn't have an easy time adjusting to the amount of people in the group, because I usually prefer working individually, or as a leader, which was not possible in this setup. But I managed to take a step back and allow the group to make some decisions, even when I didn't agree.

My subgroup had a mix of different personalities and perspectives, which attributed to a positive working environment. When disagreements occured, they were quickly resolved with mutual understanding and by following conflict resolution strategies. Consequently, the team collaboration and workflow grew stronger. We had an issue with a student not participating, but after their removal, the task delegation was easier and more equal. I believe I contributed fairly and tried to make sure everyone was involved on the same level.

From a technical perspective, I believe that my knowledge and practical experience expanded. Especially because I got the chance to develop the frontend of the system, which was not my usual area of preference. Through implementation of user interfaces, frontend components and interactive features, I gained a deeped understanding of modern frontend development processes.

The main lesson I would bring with me to my next project would be to establish clear communication between subgroups as early in the process as possible. I would try harder to integrate into the group, instead of trying to lead a workflow. And I would try to introduce some new technologies, which we could experiment with and broaden our skillsets.

Jakub Maciej Baczek

This semester was my first experience working in a group of this size, split across three subteams with distinct technical responsibilities. As Scrum Master for the IoT subteam, my role extended well beyond what I had experienced in previous projects. A significant portion of my time went into coordinating work within the team, tracking progress, and making sure the subteam's output aligned with what the other teams expected. While I found this side of the project genuinely enjoyable — understanding how all the pieces of the system fit together and interacted gave me a broader perspective than I usually get from purely technical work — it was also time consuming in a way I did not fully anticipate. Being Scrum Master in a large, distributed group often felt like needing to be available at all times, which made it difficult to separate focused technical work from coordination responsibilities.

The biggest personal challenge throughout the project was motivation. I did not find the subject matter particularly exciting, and that made it harder to stay engaged during slower periods. When the project does not feel compelling on its own terms, the motivation has to come from somewhere else — in my case, mostly from not wanting to let the subteam down.

Communication and shared understanding were also a recurring issue at the full-group level. It often felt like not everyone had a clear picture of how the overall system was supposed to look or function, which led to misalignments that could have been caught earlier with more structured cross-team communication. Looking back, I would put even more emphasis on cross-team coordination from the start, despite there already having been a fair amount of it. Making the system architecture visible and understandable to everyone — not just those who worked closely with it — would have saved time and reduced uncertainty later in the project.

On the technical side, my work covered the CI/CD pipeline for both the IoT firmware and the backend API, the IoT-facing backend endpoints, and the associated test suites. None of this was particularly difficult in isolation, but it required a solid understanding of how all components connected, which I came to appreciate more over time. The most valuable technical outcome for me personally was developing a strong understanding of DevOps and testing practices. I also had no prior experience with FastAPI or Python in a backend context, and getting to work with both gave me a new set of tools I am likely to use again. These are the takeaways I will carry into future projects regardless of how I felt about the project itself.

Tymoteusz

During this semester project I worked in the IoT team. Compared to previous projects, this one felt more complex because the IoT part was not just a separate technical component, but something that had to connect properly with the backend, MAL, and frontend. I worked on several IoT-related tasks, but the one I remember the most was trying to make the sound detector work. Unfortunately, because of the sensor limitations, we did not use it in the final project. The main problem was that it could not properly measure background sound, which was important for our idea, since noise level was one of the environmental factors connected to study conditions.

From a technical perspective, I improved my C programming skills and became more confident working with embedded systems. At the beginning, some parts of the workflow were confusing, especially the DevOps side and how the pipeline, testing, and repository structure connected together. Over time, I started to understand it better, and I think this was one of the useful learning outcomes from the project.

One challenge for me was motivation. In such a large group, it sometimes felt like not everyone was equally engaged in the project, and that was demotivating. At the same time, I think this is quite common in bigger groups, because responsibility is more spread out and it becomes easier for people to disappear into the background. Personally, time management was also difficult this semester. It was not only because of SEP, but because the whole semester required a lot of work, and I had less time for things I enjoy outside of school, such as Muay Thai and travelling.

Even with those difficulties, I actually liked working in a larger group overall. I think our Product Owner and the Scrum Master in my IoT team were good leaders, and I felt that I could count on their help when needed. That made the process less stressful, because even when something was unclear, there was usually someone I could ask.

Looking back at my own contribution, I think I completed the tasks I was responsible for and fulfilled my role in the IoT team. There were moments when some decisions changed and certain features were not used in the final system, but I still tried to stay involved and complete the work assigned to me. This project also made me more aware of how important it is to manage my time and responsibilities in a larger team, where my work can affect other parts of the system.

For future projects, I would like to plan my work earlier and communicate sooner when I am stuck or when I am unsure whether a task is still relevant. Overall, I think this project was a valuable experience before starting an internship, because it gave me a better understanding of what it is like to work in a bigger group with different roles, responsibilities, and dependencies between teams.

Cristina Matei

During this project I worked mostly on the frontend. My main work was on the profile page, dashboard, login/register flow, device connection, session start/stop, rating popup, sensor cards, and responsive design. The profile page was one of the main parts I worked on, because it included user information, password change, profile picture, and connecting a device. I also worked on the dashboard, especially with sensor values, history, session state, and rating after a study session.

At the beginning, I found it difficult to understand how all parts of the system were connected. It was not only a React project, because we also had backend, database, IoT, ML, Docker, and deployment. Many problems that looked like frontend problems were actually caused by API requests, cookies, Docker setup, or missing backend data. This made the project harder, but also more realistic.

One of the biggest learning points for me was authentication. I worked with moving the token from localStorage to cookies, protected requests, logout, rate limiting, and environment variables for secrets. I also learned more about the dashboard session flow, where the real session comes from the IoT device and the frontend only attaches to it.

Working in a large group was also challenging. Sometimes it was difficult to know who changed what, especially when several people worked on the same files. In the frontend team, communication was easier, but we still had Git merge conflicts and changes that affected each other. I learned to check dev more often, look more carefully at changes before merging, and ask earlier when something was unclear.

I also became better at debugging and testing. I used DevTools to check API requests, cookies, local storage, and errors, and I worked with frontend tests for login, dashboard states, active session handling, device connection, and rating submission.

If I worked on a similar project again, I would try to clarify important decisions earlier, especially authentication, device connection, session logic, and deployment. I would also make smaller pull requests and document decisions sooner. Overall, I learned a lot about teamwork in a bigger project.

Reflection Forward

6. REFLECTION ON SUPERVISION

6.1 The Supervision Process

6.1.1 MAL

Although we found brief moments of asking for advice in the beginning, around the elaboration phase we missed guidance that we could have sought. Towards the end, we set a goal of meeting every 1-2 weeks to never go off the path too much. Overall, we could have started earlier with asking for supervisor feedback and could have easily had less meetings towards the end if we had the correct trajectory from the start.

6.1.2 Frontend

During the inception and elaboration phase we had a couple mandatory meetings with the supervisors. They gave us insights into our architecture and design, and asked questions to motivate us to optimize our system. Besides that, the frontend group had another meeting to get some feedback from the teacher regarding UI features and functionalities. All in all, the meetings were valuable, and we couldve benefited from a larger frequency.

6.1.3 IoT

The IoT subteam had very limited supervisor interaction throughout the project. The only meeting held was to clarify whether Wi-Fi credentials needed a more user-friendly solution than the existing secrets.ini approach, which was confirmed to be acceptable. In hindsight, more regular check-ins could have helped validate technical decisions earlier and kept the subteam better aligned with overall project expectations.

6.2 Impact on the Project

6.2.1 MAL

The feedback radically changed and shaped the final approaches in the project. Not only did the advice mostly turn out to be true (predicting IoT data may be insufficient, mentioning which features could hinder or improve accuracy, etc.) but it also made us think about the problem in a more structured way. We had to justify our decisions and approaches to the supervisor, which forced us to be more critical of our own work and assumptions. This was a very valuable process that we could have benefited from even more if we had started it earlier.

6.2.2 Frontend

The meetings regarding the whole group greatly influenced some decisions. However, the individual frontend meeting did not carry much relevance, since the teacher would not give much positive or negative feedback. I believe we could have gotten more insight if we came prepared with more concrete questions.

6.2.3 IoT

The single supervisor interaction the IoT subteam had confirmed that the existing credential configuration was sufficient, which saved time that might otherwise have been spent on a more complex solution. Beyond this, supervision had little direct impact on the subteam's work or technical direction.

6.3 Lessons for Future Projects

As often, the main lesson is to be more proactive about seeking feedback and perhaps searching for efficient middle ground in many aspects. We correctly identified meetings of all to be inefficient and pointless, however, we suffered from alignment throughout the whole project. We sought supervision towards the end and missed it a lot in the middle. Overall, having a more structured start could have been of most advantage, it must, however, be noted that the semester project required utilizing the knowledge from other subjects which in the beginning made it difficult to have an accurate overview of the tasks to be accomplished and the approaches to be taken. We also had a lot of "silence and pointing

fingers", which is expected of large groups, where everyone expects someone else to bear the responsibility. Better defined roles and responsibilities as well as more structure and alignment could have fixed this problem and made the project more efficient and enjoyable.

7. CONCLUSION

7.1 Group Summary

7.1 Group Summary

The StudyHelper project became much bigger and more complicated during the semester than we first expected. In the beginning, the idea looked relatively simple — create a system that helps students understand if their study environment is good for studying. Later, the project slowly developed into a system with frontend, backend, MAL, IoT devices, databases, APIs, deployment, testing, and synchronization between multiple parts of the system.

One of the main things that defined the whole project process was teamwork between subgroups. The frontend depended on backend APIs, backend depended on IoT data, and MAL depended on sensor data and user interaction. Because of this, many tasks could not be completed independently and required constant communication between teams.

During the project, we learned that software engineering is not only about writing code. Communication, integration, version control, sprint planning, testing, debugging, and synchronization between systems became a very important part of the process. Sometimes requirements changed during development, backend structures changed, or integrations stopped working after updates, which created additional work for other teams.

The project also gave us practical experience with technologies and workflows that are used in real software development. We worked with React, APIs, authentication systems, GitHub branches, pull requests, Jira, deployment pipelines, databases, IoT communication, machine learning models, and automated testing.

Another important thing we learned was that planning something and actually implementing it are very different things. Some ideas looked simple in theory, but became much more difficult once all parts of the system needed to work together. This forced the group to constantly adapt the system structure and rethink some earlier decisions.

Working in a large group was also both useful and difficult. Different teams sometimes worked in different ways, used different priorities, or focused on different technical problems. This sometimes caused confusion or integration issues, especially during later sprints. At the same time, it gave us experience similar to real software development environments where many people work on different parts of the same system.

Overall, the project gave us valuable experience not only in programming, but also in communication, teamwork, problem solving, adaptation, and long-term project coordination. The project helped us better understand how large software systems are developed in practice and how important teamwork becomes once project complexity increases.

7.2 Recommendations

Do:

- seek supervision early and often
- establish clear communication channels and regular check-ins
- define roles and responsibilities clearly from the start
- set up CI/CD pipelines early in the development process
- use tools to track progress and accountability

Avoid:

- leaving integration until the end
- neglecting testing until late in the project
- allowing motivation to drop without addressing it
- letting conflict fester without resolution
- assuming everyone is on the same page without regular alignment checks

8. REFERENCES

- Vroom, V. H. (1964). Work and motivation .
- Darley, J. M., & Latane, B. (1968). Bystander intervention in emergencies: Diffusion of responsibility. Journal of Personality and Social Psychology , 8 (4, Pt.1), 377–383. **https://doi.org/10.1037/h0025589**
- Schwaber, K., & Sutherland, J. (2020). The Scrum Guide: The definitive guide to Scrum: The rules of the game. Scrum.org.